IdentityBoundToken v5 — Interface & Module Sketch (Prepared for controlled upgradeability 2026–2051) Core Immutable Invariants (never upgradeable) These guarantees must remain true for the life of every token: Once minted, a token can never be transferred. Every token has exactly one accountableHuman (non-zero address). Token IDs are sequential and never reused. The binding between tokenId → agent address → accountableHuman is permanent. Suggested Module Layout (UUPS / Diamond ready) IdentityBoundTokenCore ← non-upgradeable or rarely upgraded ├── SoulboundERC721 (transfer locks + _update override) ├── IdentityStorage (IdentityRecord mapping + counters) └── AccessControl + Ownable2Step Upgradeable Modules (can evolve) ├── KScoreModule (updateKScore, thresholds, risk tiers) ├── MetadataModule (tokenURI, on-chain / off-chain metadata) ├── OracleModule (DRIFT_REPORTER_ROLE management, future ZK attestation hooks) ├── GovernanceModule (threshold updates, accountableHuman changes under timelock) └── ExtensionRegistry (future: hierarchical agents, cross-chain attestations, PQ signatures) Key External Interfaces IIdentityBoundToken (minimal public surface) interface IIdentityBoundToken { // Minting (owner only) function mint( address to, address accountableHuman, uint8 initialK, string calldata metadataURI ) external returns (uint256 tokenId); // Drift reporting function updateKScore(uint256 tokenId, uint8 newK) external; // Views function getIdentity(uint256 tokenId) external view returns ( address accountableHuman, uint8 currentK, uint8 tier, // 0=ALIGNED … 3=CRITICAL uint64 mintedAt, uint64 lastUpdated, string memory metadataURI ); function getRiskTier(uint256 tokenId) external view returns (uint8); function exists(uint256 tokenId) external view returns (bool); function totalMinted() external view returns (uint256); // Admin function setRiskThresholds(uint8 alignedMin, uint8 minorMin, uint8 moderateMin) external; function updateAccountableHuman(uint256 tokenId, address newHuman) external; } Events (stable for indexers over decades) IdentityMinted(tokenId, agent, accountableHuman, initialK, metadataURI) KScoreUpdated(tokenId, oldK, newK, newTier, reporter) AccountableHumanUpdated(tokenId, oldHuman, newHuman) RiskThresholdsUpdated(alignedMin, minorMin, moderateMin) Future Extension Points (not implemented in v5 core) ZK Attestation Hook function updateKScoreWithProof(uint256 tokenId, uint8 newK, bytes calldata zkProof) external Hierarchical Agents function mintSubAgent(uint256 parentTokenId, address subAgent, …) external Cross-Chain Attestation Emit events consumable by LayerZero / Hyperlane / ZK bridges. Post-Quantum Module Replace ECDSA recovery paths only when standardized (expected ~2035+). Upgrade Safety Rules Never change the storage layout of IdentityRecord or the _identities mapping. Never remove the soulbound reverts. All new logic must live in separate modules that the core can call via a registry or diamond facets. Upgrades require a time-lock + multi-signature (or DAO) once the protocol leaves pure owner control.